

DMATH-ALGO

Analyse von Algorithmen

Herbstsemester 24

Serie 5**Lernziele**

- Sie können bei einfachen Algorithmen die Anzahl Rechenschritte im Worst-Case bestimmen.
- Sie können das Asymptotische Wachstum einer Funktion bestimmen und verstehen die Big- O , Big- Ω und Big- Θ Notationen.
- Sie kennen die wichtigsten Komplexitätsklassen.

Das Bedürfnis nach **Geschwindigkeit** hat in den letzten fünfzig Jahren einen Grossteil des Fortschritts in der Informatik vorangetrieben. Es wurden schnellere Wege entdeckt, um wichtige Probleme zu lösen: die Entwicklung von Datenstrukturen, die eine scheinbar sofortige Suche von Milliarden von Webseiten oder Hunderten von Millionen von Nutzern auf einer Social-Networking-Site unterstützen, oder die Entdeckung neuer, schnellerer Algorithmen, die praktische Probleme lösen - wie das Finden kürzerer Wege für Lieferfahrer oder das Verschlüsseln von Paketen, die über das Internet gesendet werden sollen. In dieser Serie lernen wir die **asymptotische Analyse** kennen, mit der wir die Geschwindigkeit unterschiedlicher Algorithmen für dasselbe Problem vergleichen können.

Frage 1: Warum werden Algorithmen analysiert?

Die Grundidee der Analyse besteht darin, die Wachstumsrate der Laufzeit oder des Speicherbedarfs eines Algorithmus zu betrachten - wie viel langsamer wird zum Beispiel der Algorithmus, wenn wir die Grösse der Eingabe verzehnfachen? Um die relative Leistung von Algorithmen zu analysieren, wird das langfristige Verhalten für grosse Eingaben untersucht. Irgendwelche Spezialfälle mit kleinen Eingaben, bei denen ein Algorithmus besonders gut abschneidet, sind irrelevant.

Aufgabe 1: Gegeben sei eine Funktion f , die eine Liste mit n Komponenten als Input erhält und eine bestimmte Komponente der Liste als Output zurückgibt. Wir betrachten drei Algorithmen, die diese Funktion f implementieren:

- Der Algorithmus A braucht für die Berechnung $n \log_2(n)$ Rechnungsschritte.
- Der Algorithmus B braucht für die Berechnung n^2 Rechnungsschritte.
- Der Algorithmus C braucht für die Berechnung $n! = n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1$ Rechnungsschritte.

z.B. $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$

Die Algorithmen sind auf einem Rechner implementiert, der pro Sekunde 1 000 000 000 Rechnungsschritte ausführen kann. Bestimmen Sie für die drei Algorithmen, wie viele Komponenten die Eingabeliste maximal haben darf, damit der Output innerhalb einer Minute berechnet werden kann.

Algorithmus A:

$$n \cdot \log(n) = 60 \cdot 10^9$$

für $n = 1'344'470'450$

Algorithmus B:

$$n^2 = 60 \cdot 10^9$$

für $n = 244'948$

Algorithmus C:

$$n! = 60 \cdot 10^9$$

für $n = 13$

Frage 2: Was ist ein Worst-Case Analyse?

In der Serie 1 haben wir einen Algorithmus kennengelernt, der für jede natürliche Zahl x ihre Binärdarstellung $(x)_2$ berechnet:

```
def decToBin(x):
    ① y = ''
    ① temp = x
    ① while temp != 0:
        if temp % 2 == 0: ②
            y = '0' + y
        else:
            y = '1' + y
        temp = temp // 2 ②
    ① return y
```

Aufgabe 2: Bestimmen Sie die totale Anzahl Rechnungsschritte (Zuweisungsoperationen, Vergleichsoperationen und Rechenoperationen) abhängig von der Eingabe x .

Wie viele Wiederholungen gibt es?

x	# Wiederholungen
1	1
$2^1 = 2$	2
3	2
$2^2 = 4$	3
5	3
6	3
7	3
$2^3 = 8$	4
$\vdots$	$\vdots$
2^n	$n+1$

L. J bedeutet abrunden, also z.B.

$$\lfloor 2.871 \dots \rfloor = 2$$

$$\lfloor 15.120 \dots \rfloor = 15$$

$$n+1 = \lfloor \log_2(n) \rfloor + 1$$

$$\begin{aligned} \# \text{Rechnungsschritte} &= 3 + (\lfloor \log_2(n) \rfloor + 1) \cdot 7 + 1 \\ &= \underline{\underline{4 + 7 \cdot (\lfloor \log_2(n) \rfloor + 1)}} \end{aligned}$$

Bei Algorithmus für die Funktion `decToBin()` konnten wir für jedes x eindeutig angeben, wie viele Rechnungsschritte durchgeführt werden.

Wenn aber ein Algorithmus als Input zum Beispiel eine Liste der Länge n verarbeitet, hängt die Anzahl Rechnungsschritte im Allgemeinen auch von den Werten ab, die in so einer Liste gespeichert sind.

Definition 1: Bei einer *Worst-Case-Analyse* wird in Abhängigkeit von der *Grösse* n des Inputs die maximale Anzahl Rechnungsschritte angegeben, die zur Berechnung nötig sein können.

Aufgabe 3: Bestimmen Sie für den folgenden linearen Suchalgorithmus die totale Anzahl Rechnungsschritte für eine Liste der Länge n , die im Worst-Case nötig sind:

```
def linearSearch(Liste, Element):  
    for i in range(len(Liste)): ①  
        if Liste[i] == Element: ② return True  
    ③ return False ④
```

Best-Case: $1 + 2 = 3$

Worst-Case: $n \cdot (1 + 1) + 1 = \underline{\underline{2n + 1}}$

$2n + 1$ ist in $O(n)$, weil:

$$2n + 1 \leq 2n + n = 3 \cdot n \quad \text{für } n \geq 1$$

$2n + 1$ ist in $\Omega(n)$, weil:

$$2n + 1 \geq 2n + 0 = 2 \cdot n \quad \text{für } n \geq 0$$

Also: Worst-Case von `linearSearch`
ist in $\Theta(n)$

Bereits die Worst-Case-Analyse eines einfachen Sortieralgorithmus kann anspruchsvoll sein:

Aufgabe 4: Bestimmen Sie für den folgenden Sortieralgorithmus die totale Anzahl Rechnungsschritte für eine Liste der Länge n , die im Worst-Case nötig sind. Was ist die kleinste Anzahl Rechnungsschritte (Best-Case), die für alle Listen der Länge n nötig sind?

```
def selectionSort(Liste):
    ① for i in range(len(Liste)):
        ① minIndex = i
        for j in range(i+1, len(Liste)): ①
            ① if Liste[j] < ① Liste[minIndex]: minIndex = ① j
            # swap Liste[i] and Liste[minIndex]
            ② temp = Liste[i]
            ① Liste[i] = Liste[minIndex]
            ① Liste[minIndex] = temp
```

n mal

$i=0$ $(n-1)$ mal j -Schleife
 $i=1$ $(n-2)$ mal j -Schleife
 $i=2$ $(n-3)$ mal j -Schleife
 $\vdots$
 $i=n-2$ 1 mal j -Schleife
 $i=n-1$ 0 mal j -Schleife

$$\text{Worst-Case : } n \cdot 5 + (n-1) \cdot 3 + (n-2) \cdot 3 + (n-3) \cdot 3 + \dots + 1 \cdot 3$$

$$\begin{aligned}
 * & (n-1) + (n-2) + \dots + 1 = 5 \\
 & 1 + 2 + \dots + (n-1) = 5 \\
 n + n + \dots + n &= 25 \\
 (n-1) \cdot n &= 25 \\
 \frac{(n-1) \cdot n}{2} &= 5
 \end{aligned}$$

$$\begin{aligned}
 &= 5n + 3 \cdot [(n-1) + (n-2) + \dots + 1] \\
 &= 5n + 3 \cdot \frac{n(n-1)}{2} \\
 &= 5n + \frac{3}{2}n^2 - \frac{3}{2}n = \frac{3}{2}n^2 + \frac{7}{2}n
 \end{aligned}$$

$$\text{Best-Case : } 5n + 2 \cdot \frac{n(n-1)}{2} = \underline{\underline{n^2 + 4n}}$$

selectionSort ist in $\Theta(n^2)$.

Aufgabe 5: Bestimmen Sie für den folgenden Sortieralgorithmus die totale Anzahl Rechnungsschritte für eine Liste der Länge n , die im Worst-Case nötig sind. Was ist die kleinste Anzahl Rechnungsschritte (Best-Case), die für alle Listen der Länge n nötig sind?

```
def insertionSort(Liste):
    for i in range(1, len(Liste)):
        j = i
        while j > 0 and Liste[j] < Liste[j-1]:
            temp = Liste[j]
            Liste[j] = Liste[j-1]
            Liste[j-1] = temp
            j = j-1
```

Diagramm zur Analyse der Schleifen:

- Die `for`-Schleife wird $(n-1)$ mal ausgeführt (blau markiert).
- Innerhalb der `for`-Schleife wird `j = i` gesetzt.
- Die `while`-Schleife wird für jedes i von 1 bis $n-1$ ausgeführt.
 - Für $i=1$: 1 mal `j`-Schleife
 - Für $i=2$: 2 mal `j`-Schleife
 - Für $i=3$: 3 mal `j`-Schleife
 - ...
 - Für $i=n-1$: $(n-1)$ mal `j`-Schleife
- Innerhalb der `while`-Schleife sind 4 Operationen (Zuweisungen/Vergleiche) markiert (rot markiert).

$$\begin{aligned}
 \text{Worst-Case} &: (n-1) \cdot 4 + 1 \cdot 6 + 2 \cdot 6 + 3 \cdot 6 + \dots + (n-1) \cdot 6 \\
 &= 4(n-1) + 6 \cdot (1 + 2 + 3 + \dots + (n-1)) \\
 &= 4(n-1) + 6 \cdot \frac{n(n-1)}{2} \\
 &= 4n - 4 + 3n(n-1) \\
 &= \underline{\underline{3n^2 + n - 4}}
 \end{aligned}$$

$$\text{Best-Case} : (n-1) \cdot 4 = \underline{\underline{4n - 4}}$$

Worst-Case von insertionSort ist in $\Theta(n^2)$

Best-Case von insertionSort ist in $\Theta(n)$

```
def bubbleSort(Liste):
    ① for i in range(len(Liste)):
        ① for j in range(len(Liste)-1):
            ① if Liste[j] > Liste[j+1]:
                temp = Liste[j]
                Liste[j] = Liste[j+1]
                Liste[j+1] = temp
```

① $i=0$ $(n-1)$ mal j-Schleife
 ① $i=1$ $(n-1)$ mal j-Schleife
 ① $i=2$ $(n-1)$ mal j-Schleife
 ...
 ① $i=n-1$ $(n-1)$ mal j-Schleife

n mal $(n-1)$ mal

$$\begin{aligned}\text{Worst-Case: } & n \cdot 1 + n \cdot (n-1) \cdot 5 \\ &= n + 5n(n-1) \\ &= \underline{\underline{5n^2 - 4n}}\end{aligned}$$

Best-case: $u \cdot 1 + u \cdot (u-1) \cdot 2$
 $= u + 2u(u-1)$
 $= \underline{2u^2 - u}$

bubbleSort ist in $\Theta(n^2)$.

Aufgabe 7: Bestimmen Sie für den folgenden Suchalgorithmus die totale Anzahl Rechnungsschritte für eine *geordnete* Liste der Länge n , die im Worst-Case nötig sind. Was ist die kleinste Anzahl Rechnungsschritte (Best-Case), die für alle Listen der Länge n nötig sind?

```
def binarySearch(Liste, Element):
    ① left = 0
    ① ① right = len(Liste)-1
    ① while left <= right:
        middle = (left + right)//2  ① ① ✓
        if Liste[middle] == Element: ① ✓
            return True ①
        elif Liste[middle] > Element: ① ✓
            right = middle - 1 ① ①
        else: ① ①
            left = middle + 1 ① ①
    ① return False
```

entweder oder

Worst-Case: (Element ist nicht in der Liste)

n	#while	
1	1	
2	2	
3	2	
4	3	
5	3	
6	3	
7	3	
8	4	
...	...	
2^n	$n+1 = \lfloor \log_2(n) \rfloor + 1$	weil die rechte Hälfte nicht kleiner ist als die linke Hälfte, nehmen wir im worst-case den else-Fall.

$$\begin{aligned}
 & 5 + \text{\#while} \cdot 7 \\
 &= 5 + (\lfloor \log_2(n) \rfloor + 1) \cdot 7 \\
 &= \underline{\underline{12 + 7 \cdot \lfloor \log_2(n) \rfloor}}
 \end{aligned}$$

Best-Case : $4 + 4 = \underline{\underline{8}}$

Worst-Case von binarySearch ist in $\Theta(\log(n))$.

Frage 3: Was ist eine asymptotische Analyse?

Das Zählen der genauen Anzahl der Rechnungsschritte eines Algorithmus ist oft sehr schwierig. Zudem stellt sich die Frage, ob die exakte Angabe der Rechnungsschritte überhaupt sinnvoll ist. Weil die Rechner eine gewaltige Rechenpower haben, dürfte es keinen Unterschied ausmachen, ob ein Algorithmus im Worst-Case $n^2 + 3n + 3$ oder $n^2 + 5n + 7$ Rechnungsschritte benötigt. Vielmehr wäre es sinnvoll Algorithmen in **Klassen** unterschiedliche **Rechenzeitkomplexität** oder **Speicherplatzkomplexität** einzuteilen.

Algorithmen, die zu unterschiedlichen Komplexitätsklassen gehören, sollen sich wesentlich voneinander unterscheiden.

Definition 2: Seien f und g zwei Funktionen, welche die Rechnungsschritte zweier Algorithmen angeben. Wir sagen, dass die Funktion f

- **nicht stärker wächst** als die Funktion g , wenn für alle Input-Werte x grösser als ein Startwert x_0 gilt, dass

$$f(x) \leq C \cdot g(x) \text{ für eine feste Konstante } C.$$

Kurz bezeichnen wir dies mit **$f(x)$ ist in $O(g(x))$** (Big-O Notation).

- **nicht langsamer wächst** als die Funktion g , wenn für alle Input-Werte x grösser als ein Startwert x_0 gilt, dass

$$f(x) \geq c \cdot g(x) \text{ für eine feste Konstante } c.$$

Kurz bezeichnen wir dies mit **$f(x)$ ist in $\Omega(g(x))$** (Big-Ω Notation).

- **asymptotisch gleich wächst** wie die Funktion g , wenn

$$f(x) \text{ in } O(g(x)) \text{ und in } \Omega(g(x)) \text{ ist.}$$

Kurz bezeichnen wir dies mit **$f(x)$ ist in $\Theta(g(x))$** (Big-Θ Notation).

Um zu zeigen, dass eine Funktion eine dieser Eigenschaften hat, müssen wir Zeugen x_0 und C oder x_0 und c angeben und mit einer Rechnung begründen, dass die Ungleichung stimmt.

Aufgabe 8: Begründen Sie, dass

a) $f(x) = 4x^5 + 15x^3 - 10x^2 - 8$ in $O(x^6)$ ist.

$$\begin{aligned} &\leq 4x^6 + 15x^6 - 0 - 0 \\ &= 19 \cdot x^6 \quad \text{für } x \geq 1. \end{aligned}$$

b) $f(x) = 4x^5 + 15x^3 - 10x^2 - 8$ in $\Omega(x^3)$ ist.

$$\begin{aligned} &\geq 4x^3 + 15x^3 - 10x^2 - 8 \\ &\geq 4x^3 + 15x^3 - 10x^3 - 8x^3 \\ &= 1 \cdot x^3 \quad \text{für } x \geq 1. \end{aligned}$$

c) $f(x) = 5x^2 - 2x + 8$ in $\Theta(x^2)$ ist.

$$\begin{aligned} &\leq 5x^2 - 0 + 8 \cdot x^2 = 13x^2 \quad \text{für } x \geq 1. \\ f(x) &\geq 5x^2 - 2x^2 + 0 = 3x^2 \quad \text{für } x \geq 1. \end{aligned}$$

d) $f(x) = 5x \log_2(3x^x) + 3x^2$ in $O(x^2 \log(x))$ ist.

$$\begin{aligned} &= 5x \cdot (\log_2(3) + \log_2(x^x)) + 3x^2 \\ &= 5x \cdot \log_2(3) + 5x \cdot x \log_2(x) + 3x^2 \\ &\leq 5x \cdot \log_2(x) + 5x^2 \log_2(x) + 3x^2 \quad \text{für } x \geq 3 \\ &\leq 5x^2 \log_2(x) + 5x^2 \log_2(x) + 3x^2 \log_2(x) \\ &= 13 \cdot x^2 \log(x) \end{aligned}$$

e) $f(x) = (8x + x^2 \log_{10}(5^x)) \log_{10}(10x)$ in $O(x^3 \log(x))$ ist.

$$\begin{aligned} &= (8x + x^2 \cdot x \log_{10}(5)) \cdot \log_{10}(10x) \\ &\leq (8x + x^3 \cdot 1) \cdot \log_{10}(10x) \\ &\leq (8x + x^3) \cdot \log_{10}(x^2) \quad \text{für } x \geq 10 \\ &= (8x + x^3) \cdot 2 \cdot \log_{10}(x) \\ &\leq (8x^3 + x^3) \cdot 2 \cdot \log_{10}(x) \\ &= 18 \cdot x^3 \log_{10}(x) \end{aligned}$$

Frage 4: Welche Komplexitätsklassen sind wichtig?

Mit der Big-O Notation können wir die wichtigsten Komplexitätsklassen für die asymptotische Analyse von Algorithmen beschreiben. Sei n die Grösse des Inputs. Wenn die Anzahl der Rechnungsschritte $f(n)$ in den Komplexitätsklassen

$$O(1) \subset O(\log(n)) \subset O(n) \subset O(n \log(n)) \subset O(n^2) \subset \dots \subset O(n^k) \subset \dots$$

ist, hat der Algorithmus ein **polynomielles Wachstum**. Probleme, die mit einem solchen Algorithmus gelöst werden können, heissen **real lösbar**. Wenn die Anzahl der Rechnungsschritte $f(n)$ in den Komplexitätsklassen

$$O(2^n) \subset O(3^n) \subset O(4^n) \subset \dots \subset O(a^n) \subset \dots$$

ist, hat der Algorithmus ein **exponentielles Wachstum**. Probleme, die nur mit einem solchen Algorithmus gelöst werden können, heissen **real unlösbar**.

Beispiele:

- Ein Speicherzugriff ist in $\Theta(1)$.
- Die binäre Suche in einer Liste mit n Komponenten ist in $\Theta(\log(n))$.
- Die lineare Suche in einer Liste mit n Komponenten ist in $\Theta(n)$.
- Das Sortieren einer Liste mit n Komponenten nach dem Mergesort Verfahren von John von Neumann ist in $\Theta(n \log(n))$.
- Das Sortieren einer Liste mit n Komponenten mit den meisten klassischen Verfahren ist in $\Theta(n^2)$.
- Das Multiplizieren zweier $n \times n$ Matrizen in der Linearen Algebra mit naivem Vorgehen ist in $\Theta(n^3)$.
- Das SAT Problem für eine Aussage mit n aussagenlogischen Variablen mit naivem Vorgehen ist in $\Theta(2^n)$.